

這是一份針對 **Chapter 2 (Emulation: Interpretation and Binary Translation)** 的詳盡考試重點整理，涵蓋了模擬技術的核心概念、各類解譯與翻譯技術的比較、實作挑戰及優化手段。

一、 模擬 (Emulation) 核心定義

- **定義**：在具有不同介面與功能的系統（Target ISA）上，實作另一系統（Source ISA）的介面與功能。
- **組成部分**：模擬不僅包含指令集，還涉及暫存器狀態、記憶體架構、Trap（陷阱）與中斷架構。

二、 解譯技術 (Interpretation Methods) 的演進與比較

解譯器的核心是操作 **Source Context Block**（包含 SPC、暫存器、狀態碼等）。

技術名稱	運作機制	優點	缺點
解碼與發送 (Decode-and-Dispatch)	中央循環 (Loop) 負責擷取、解碼，透過 <code>switch</code> 發送至對應常式。	記憶體需求極低；啟動速度最快（無需預處理）。	穩態效能最慢；重複解析相同指令；資料快取壓力大；分支預測效率低。
間接執行緒 (Indirect Threaded)	將發送代碼附加在每個常式結尾，透過發送表間接跳轉。	消除中央循環的分支，效能略優於前者。	仍有查表的記憶體存取開銷。
預先解碼 (Predecoding)	執行前先將指令解析成較易處理的中間形式 (Intermediate Form, IF)。	減少重複解析指令的開銷；簡化常式邏輯。	啟動較慢；需額外空間存 IF 映像檔。
直接執行緒 (Direct Threaded)	在 IF 中直接將操作碼替換為模擬常式的「實際位址」。	消除發送表查表開銷，穩態效能最佳。	可攜性差 （依賴常式在記憶體的位置）。

技術名稱	運作機制	優點	缺點

三、處理複雜指令集 (CISC, 如 IA-32) 的策略

- **分階段解譯**：第一階段填寫指令模板 (Template)，第二階段根據類型發送。
- **優化常見案例 (Common Case Fast)**：
 - IA-32 常見情況：無前綴、單字節操作碼、暫存器操作數。
 - 針對常見指令使用精簡的發送路徑，複雜指令才交由中央解譯。
- **軟體管線化 (Software Pipelining)**：在 IA-32 高效能解譯器中，可重疊「目前指令執行」與「下一指令載入/長度查詢」以掩蓋延遲。

四、二進位翻譯 (Binary Translation, BT)

- **核心理念**：將 Source ISA 的指令區塊轉換成 Target ISA 的原生指令並執行。
- **狀態映射 (State Mapping)**：
 - 將客體暫存器直接映射到主體硬體暫存器（取代記憶體 context block），可大幅提升速度。
- **挑戰：代碼發現問題 (Code Discovery Problem)**：
 - 原因：變長指令（CISC）難以劃分邊界、指令與資料混雜（如跳轉表）、間接跳轉目標不可知。
 - **解決方案：動態翻譯 (Dynamic Translation)**：僅在程式執行到該路徑時才進行「增量 (Incremental)」翻譯。

五、動態翻譯系統架構 (Dynamic Translation System)

1. **模擬管理器 (Emulation Manager, EM)**：負責高層級控制流切換。
2. **解譯器 (Interpreter)**：負責初次執行、代碼發現與剖析。

3. **代碼快取 (Code Cache)**：儲存已翻譯好的目標代碼區塊（通常以 **Dynamic Basic Block** 為單位）。
 4. **對應表 (Map Table)**：雜湊表結構，記錄 **SPC** ↔ **TPC** 的映射關係，用於快取命中偵測。
 5. **存根 (Stub)**：位於翻譯塊結尾，當目標未翻譯時，負責將控制權與 **SPC** 傳回 **EM**。
-

六、優化技術 (Optimizations)

- **翻譯鏈結 (Translation Chaining)**：將翻譯塊直接串接，避免頻繁透過 **EM** 進行查表，模擬執行緒解譯的概念。
 - **軟體間接跳轉預測**：在代碼中內聯 (Inline) 緩存常見的目標 **SPC**，避免查詢 **Map Table**。
 - **影子棧 (Shadow Stack)**：保存返回位址的 **SPC** ↔ **TPC** 映射，加速函數返回。
 - **延遲狀態評估 (Lazy Evaluation)**：針對狀態碼 (Condition Codes)，僅在後續指令真的需要時才進行計算，以減少頻繁更新狀態的開銷。
-

七、效能權衡 (Performance Tradeoffs)

- **公式**：總執行時間 = $S + NT$ (S : 啟動成本, N : 執行次數, T : 穩態執行時間)。
 - **決策點**：
 - 解譯： $S \approx 0$ ， T 很高。適合執行次數少的冷點代碼。
 - 二進位翻譯： S 很高， T 很低。適合頻繁執行的「熱點 (Hotspots)」代碼。
 - **階段式模擬 (Staged Emulation)**：
 - 1. 先以解譯執行並剖析 (Profiling)。
 - 2. 若執行次數達到閾值 (Threshold)，則啟動二進位翻譯並快取。
 - 3. 極熱點可進一步進行超區塊 (Superblock) 優化。
-

八、相同 ISA 的模擬應用 (Same-ISA Emulation)

即使 Guest 與 Host ISA 相同，模擬仍有其用處：

1. **模擬 (Simulation)**：如 **Shade** 系統，用於收集執行追蹤 (Trace)。
2. **OS 呼叫模擬**：跨作業系統執行程式。
3. **特權操作管理**：系統虛擬機攔截關鍵指令。
4. **程式護送 (Program Shepherding)**：基於安全性監控跳轉目標。
5. **動態二進位最佳化**：對既有二進制檔案進行執行時優化。

ch3

這份針對 **Chapter 3 (Process Virtual Machines)** 的詳盡考試重點整理，涵蓋了程序虛擬機器的架構、相容性、狀態映射、指令與作業系統模擬以及快取管理。

一、程序虛擬機 (Process VM) 基本概念與架構

- **定義**：在主體 (Host) 作業系統之上的 ABI 介面提供一個虛擬環境，支援單個客體 (Guest) 應用程式執行。
 - **核心組件**：
 - **Loader (載入器)**：載入客體二進位檔。
 - **Initialization (初始化)**：分配記憶體、設置信號處理常式 (Signal Handlers)。
 - **Emulation Engine (模擬引擎)**：包含詮釋器 (Interpreter) 與翻譯器 (Translator)。
 - **Code Cache (代碼快取)**：存放翻譯後的目標代碼。
 - **Code Cache Manager**：管理快取替換與清理。
 - **Profile Database**：收集執行數據以引導最佳化。
 - **OS Call Emulator**：將客體系統呼叫轉換為主體作業系統的操作。
-

二、相容性 (Compatibility) 的層級與條件

- **定義**：客體在主體上的行為準確度。排除了效能差異，僅專注於功能的正確性。

- **相容性層級：**
 - **本質相容性 (Intrinsic Compatibility)：**對**所有**程式與資料 100% 準確，達到完全透明。
 - **外在相容性 (Extrinsic Compatibility)：**僅針對**特定子集**（如特定編譯器產生的程式或特定庫）保證相容。
 - **充分相容條件：**
 - 狀態等效性不需在每條指令維持，只需在**作業系統控制轉移點** (OS Control Transfer Points，如系統呼叫、Trap) 維持即可。
 - 從 OS 回復執行時，客體狀態必須與原生平台產生的結果一致。
-

三、狀態映射 (State Mapping)

- **暫存器映射：**
 - **主體暫存器多：**客體暫存器可直接映射到主體暫存器。
 - **主體暫存器不足：**需在記憶體中維護 **Register Context Block**。切換模式時需執行載入/卸載 (Load/Unload)。
 - **記憶體位址空間映射：**
 - **軟體轉換表：**類似分頁表，將客體區塊映射到非連續的主體區域，靈活但速度慢。
 - **直接映射 (Direct Mapping)：**
 - **固定位移 (Fixed Offset)：**客體位址 + 固定常數。
 - **零位移 (Zero Offset)：**客體位址與主體虛擬位址完全一致，效能最佳。
 - **位址空間共享：**Runtime 軟體必須與客體程式共享同一個主體虛擬位址空間。
-

四、記憶體架構模擬與保護

- **保護機制：**利用主體 OS 的 `mprotect()` 與 `SIGSEGV` 信號來實現 R/W/E 權限檢查。
- **分頁大小不匹配：**
 - 若客體分頁 < 主體分頁，多個客體分頁會擠在同一個主體分頁。

- **解法**：採取保守權限策略（取交集最嚴格者），並由 Runtime 處理額外的 Trap。
 - **自我修改代碼 (Self-Modifying Code, SMC)**：
 - **基本做法**：將存放翻譯代碼的原始頁面設為「唯讀」。
 - **觸發流程**：寫入行為觸發 Trap → Runtime 清除對應的快取翻譯塊 → 暫時改為 R/W → 以詮釋模式執行該區塊 → 回復唯讀保護。
 - **保護 Runtime 記憶體**：
 - 在**模擬模式**下，利用硬體保護將 Runtime 的數據/代碼設為不可存取 (No-access)，代碼快取設為唯執行 (Execute-only)。
-

五、指令模擬與效能權衡

- **效能公式**：Total Time=S+NT (S: 啟動成本, N: 執行次數, T: 穩態執行時間)。
 - **階段式模擬 (Staged Emulation)**：
 1. **詮釋階段**：啟動快，但 T 高。同時進行熱點偵測。
 2. **翻譯階段**：當執行次數超過閾值 (Threshold，如 50 次)，翻譯為基本區塊並快取。
 3. **最佳化階段**：極熱點會被組成超區塊 (Superblocks) 並進行深度最佳化。
-

六、例外與作業系統模擬

- **精確狀態恢復 (Precise State Recovery)**：當發生 Trap 時，需透過**側表 (Side Tables)** 定位 SPC，並視需求重新詮釋原始代碼以重現精確的客體狀態。
- **系統呼叫模擬**：
 - **相同 OS**：透過 **Wrapper (包裝程式)** 進行參數轉換後直接呼叫主體系統呼叫。
 - **不同 OS**：需逐一模擬語義，難度極高且常不完整（如 Wabi 系統僅支援部分 Word/Excel 功能）。
- **回呼函式 (Callbacks)**：Runtime 需攔截 OS 的發送常式 (Dispatch Routine)，確保執行使用者回呼時仍受控於 VM。

七、代碼快取管理 (Code Cache Management)

- **替換策略**：由於區塊大小不等，常用 **FIFO** 或 **LRU** 算法。
- **清理機制**：
 - **滿了就清 (Flush when full)**：簡單但會導致短暫效能下降。
 - **工作集偵測 (Working Set Detection)**：偵測到大量新翻譯需求時主動清理舊翻譯。

ch4

這是一份針對 **Chapter 4 (Dynamic Binary Optimization)** 的詳盡考試重點整理。本章接續 Chapter 3 的基礎，專注於如何透過動態技術提升模擬效能。

一、動態程序行為 (Dynamic Program Behavior)

優化的基礎在於程式行為的高度可預測性。

- **控制流預測 (Control Flow Predictability)**：
 - 分支指令 (Conditional Branches) 通常有很強的傾向性（大多 Taken 或大多 Not Taken）。
 - **後向分支**（如迴圈結尾）通常是 **Taken**；**前向分支**（如錯誤檢查）通常是 **Not Taken**。
 - 約 63% 到 98% 的分支會重複上一次的執行結果。
 - **數值不變性 (Value Constancy)**：許多指令在執行時會重複產生相同的結果值，這提供了「特化 (Specialization)」優化的機會。
-

二、剖析技術 (Profiling)

剖析是收集程序執行統計數據的過程，用以引導優化決策。

- **收集方式：**
 - **檢測式 (Instrumentation-based)：**在代碼中插入「探針 (Probes)」來計數。準確但開銷較高。
 - **採樣式 (Sampling)：**利用定時中斷檢視硬體狀態（如 PC）。開銷低，適合長時間監控。
 - **剖析類型：**
 - **熱點偵測 (Hotspot Detection)：**找出執行頻繁的區域（如執行超過一定閾值的基本區塊）。
 - **節點剖析 (Node Profiling)：**統計基本區塊執行次數。
 - **邊緣剖析 (Edge Profiling)：**統計分支跳轉的路徑頻率，這對形成大型翻譯塊至關重要。
-

三、 優化翻譯塊 (Optimizing Translation Blocks)

為了擴大優化範圍，必須跨越基本區塊（Basic Block）的邊界。

- **改善局部性 (Locality)：**
 - **空間局部性：**將頻繁執行的塊排在一起，減少快取行（Cache Line）中不常用代碼佔用的空間。
 - **時間局部性：**重複執行的代碼保持在快取中。
 - **主要區塊類型：**
 - **追蹤 (Traces)：**一連串連續執行的基本區塊。
 - **超區塊 (Superblocks)：**單一入口、多個出口的區塊。透過「跳轉取反」和「複製代碼」來消除副作用進入點。
 - **樹群 (Tree Groups)：**包含多條路徑但維持單一入口的結構。
 - **程序內聯 (Procedure Inlining)：**將被呼叫函數的代碼直接嵌入呼叫處，消除 Call/Return 的開銷及暫存器存取負擔。
-

四、 優化框架與相容性 (Optimization Framework & Compatibility)

- **優化流程**：收集基本區塊 → 轉換為**中間形式 (Intermediate Form, IF)** → 指令排程與優化 → 生成目標代碼 → 加入**補償碼 (Compensation Code)**。
 - **中間形式 (IF)**：通常採用**單一賦值格式 (Single Assignment)**，以簡化依賴關係分析。
 - **相容性限制**：
 - **陷阱相容性 (Trap Compatibility)**：優化不可移除原本會觸發異常（如溢位）的指令，除非能保證狀態等效。
 - **精確狀態恢復**：若優化後的代碼發生異常，必須能回溯到最近的**檢查點 (Checkpoints)** 並重現客體狀態。
-

五、指令重排 (Code Reordering)

調整指令順序以減少管線停頓（Stalls）並提升並行度。

- **指令分類**：
 1. **暫存器更新 (reg)**：可透過臨時暫存器「撤銷」操作。
 2. **記憶體更新 (mem)**：如 Store，難以撤銷，限制較多。
 3. **分支 (br)**：確定控制流。
 4. **進入點 (join)**：跳轉目標進入區塊的地點。
 - **重排規則 (Table 4.1)**：
 - 暫存器更新指令可以移到分支之前，但若分支未中，可能需要補償碼或臨時變數來保護舊值。
 - Store 指令（mem）通常不能移到分支或可能觸發 Trap 的指令之前。
-

六、常見優化手段 (Basic Optimizations)

這些經典編譯器技術在動態翻譯中同樣有效：

1. **常數傳播 (Constant Propagation)**：將已知數值代入後續指令。
2. **常數摺疊 (Constant Folding)**：在編譯時先行計算常數運算（如 $2+6$ 變 8）。

3. **強度削減 (Strength Reduction)**：將昂貴運算換成廉價運算（如乘法換成位移）。
 4. **複製傳播 (Copy Propagation)**：消除不必要的寄存器間搬移。
 5. **死碼刪除 (Dead-Assignment Elimination)**：移除不再被使用的賦值。
 6. **代碼下沉 (Code Sinking)**：將部分路徑才需要的運算移入該分支路徑中。
 7. **循環不變量外提 (Loop Invariant Hoisting)**：將迴圈內不會變動的運算移到迴圈外執行一次。
-

七、相同 ISA 優化系統 (Same-ISA Optimizers)

即使 Guest 與 Host ISA 相同，動態優化仍具價值（如 **HP Dynamo**）。

- **優點：**
 - 無語義不匹配問題（如狀態碼相同）。
 - **退出機制 (Bailout)**：若優化無效，可隨時回歸執行原始二進位碼。
 - **代碼補丁 (Patching)**：直接在原始代碼中插入跳轉指令到優化後的快取塊，避免頻繁查表。
 - **挑戰：**資源競爭。優化器與客體程式共享相同的暫存器與記憶體空間，缺乏多餘的暫存器作為臨時暫存。
-

八、階段式優化 (Staged Optimization)

為了平衡優化成本與收益，通常採取多階段策略：

1. **解譯階段**：快速啟動並收集剖析數據。
2. **簡單編譯 (Baseline/JIT)**：將熱點轉換為基本區塊並快取。
3. **深度優化**：針對極熱點進行跨塊優化、內聯與指令重排。

這是一份針對 **Chapter 8 (System Virtual Machines)** 的詳盡考試重點整理。本章的核心在於如何讓單一硬體平台同時支援多個完整的客體作業系統 (Guest OS) 及其應用程式。

一、系統虛擬機 (System VM) 的核心定義與動機

- **定義**：透過虛擬機器監視器 (VMM) 管理硬體資源，使多個系統映像 (System Images) 能共享同一組物理資源，且每個客體 OS 都認為自己擁有獨立的硬體。
 - **虛擬機器監視器 (VMM)**：擁有硬體最終控制權的軟體層，負責攔截客體 OS 的特權指令並確保隔離性。
 - **主要應用**：
 - **資源管理**：實現多工與時間分片 (Time-sharing)。
 - **系統轉移與相容性**：過渡到新 OS 或執行舊版 (Legacy) 程式。
 - **安全與隔離**：將損毀或受攻擊的風險限制在單一 VM 內。
 - **開發與測試**：在不崩潰主機的情況下測試新系統軟體。
 - **系統封裝**：檢查點 (Checkpointing) 與狀態傾印 (Dumping)，利於環境遷移。
-

二、虛擬化的理論基礎：Popek & Goldberg 定理

這是本章最常考的理論重點，定義了 ISA 是否能「高效虛擬化」的充足條件。

1. 指令分類 (Instruction Classification)：

- **特權指令 (Privileged)**：在使用者模式 (User Mode) 下執行會產生 **Trap** 的指令。
- **敏感指令 (Sensitive)**：
 - **控制敏感 (Control-sensitive)**：企圖修改系統資源配置 (如記憶體映射或計時器)。
 - **行為敏感 (Behavior-sensitive)**：其執行結果取決於當前資源狀態 (如處理器模式)。

- **無害指令 (Innocuous)**：不屬於上述兩類的指令。

2. 定理一 (Theorem 1)：

- **條件**：當一個 ISA 的**敏感指令集是特權指令集的子集**時，該架構是可高效虛擬化的。
- **邏輯**：這確保了所有企圖影響系統狀態的操作都會自動觸發 Trap，交由 VMM 攔截並處理，其餘普通指令則直接由硬體執行。

3. 關鍵指令 (Critical Instructions)：

- 指那些「**敏感但非特權**」的指令。它們在虛擬機中執行時**不會觸發 Trap**，會導致執行結果錯誤（例如 IA-32 的 `POPF` 指令）。

三、處理器與指令虛擬化技術

針對非虛擬化架構（如 IA-32）的解決方案：

- **二進位翻譯 (Binary Translation, BT)**：
 - VMM 掃描指令流，遇到特殊或關鍵指令時，將其翻譯為等效的目標指令序列（例如修改虛擬 CPU 的標誌位元）。
- **補丁技術 (Patching / Scanning and Patching)**：
 - 在執行前掃描基本區塊 (Basic Block)，將發現的關鍵指令替換為 Trap 或 Jump 指令，強制控制權轉回到 VMM。
- **模擬碼快取 (Caching Emulation Code)**：
 - 為了避免重複翻譯的開銷，將翻譯後的模擬常式存入 Code Cache。
- **遞歸虛擬化 (Recursive Virtualization)**：
 - 在 VM 中再跑一個 VMM。
 - **定理二 (Theorem 2)**：必須滿足系統可虛擬化，且 VMM 本身不具備時間依賴性 (Timing Dependences)。

四、資源虛擬化 (Memory & I/O)

1. 記憶體虛擬化：

- **陰影分頁表 (Shadow Page Tables)**：客體 OS 維護「虛擬到真實 (V-to-R)」的對應，VMM 則維護「虛擬到物理 (V-to-P)」的陰影表，硬體直接使用後者進行地址轉換。
- **記憶體氣球 (Ballooning)**：透過客體內的驅動程式強迫 OS 釋放不必要的記憶體，以便重新分配物理資源。

2. I/O 虛擬化：

- **設備類型**：專用 (Dedicated)、分割 (Partitioned)、共享 (Shared) 或列隊 (Spooled)。
- **層次**：可在系統呼叫層、設備驅動層或 I/O 匯流排層 (操作層級) 進行攔截模擬。

五、硬體輔助技術 (Hardware Assists)

1. IBM IEF (Interpretive Execution Facility)：

- **核心指令：SIE (Start Interpretive Execution)**。
- **運作**：VMM 設置狀態描述表後執行 SIE，硬體直接處理大部分特權指令，僅在必要時觸發攔截 (Interception) 退出回 VMM。

2. Intel VT-x (Vanderpool)：

- **VMX 模式**：分為 **Root 操作** (供 VMM) 與 **Non-Root 操作** (供 Guest VM)。
- **優點**：客體作業系統核心可以直接運行在 **Ring 0**，解決了傳統虛擬化的特權級壓縮 (Ring Compression) 問題。
- **VMCS (Virtual Machine Control Structure)**：存放 VM 狀態與執行控制位元的硬體資料結構，自動管理狀態載入/卸載。

六、虛擬機類型與特性

- **原生型 (Native) vs. 裝載型 (Hosted)**：
 - **Native**：VMM 直接跑在硬體上 (如 ESX)。
 - **Hosted**：VMM 跑在主機 OS 上 (如 VMware Workstation)，可直接利用主機的驅動程式。

- **即時遷移 (Live Migration) :**
 - 流程：建立基準拷貝 → 迭代傳輸變動 (Dirty) 頁面 → 凍結 VM → 傳送最終狀態與處理器狀態 → 在目標端恢復執行。
- **快照 (Snapshots) 與複製 (Clones) :**
 - 方便進行研究測試，提升系統開發效率。